

Universidade Federal de Uberlândia

Aula 13: Busca em árvore de Monte Carlo e a família AlphaZero

Busca baseada em simulação • UCT • AlphaGo, AlphaZero e MuZero

Prof. Marcelo Keese Albertini
Universidade Federal de Uberlândia
adaptado de Emma Brunskill, CS234 (Stanford)

2026



Plano de hoje

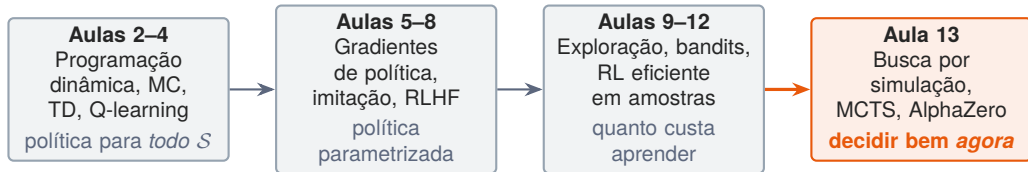
1. **Busca baseada em simulação** — a mudança de perspectiva: em vez de calcular uma política para *todo* o espaço de estados, gastar computação para decidir bem *agora*.
2. **Busca de Monte Carlo simples e árvores expectimax** — o que se ganha e por que a enumeração explode.
3. **MCTS e UCT** — as quatro fases, o bandit em cada nó, garantias e limitações.
4. **AlphaGo** → **AlphaGo Zero** → **AlphaZero** → **MuZero** — a leitura moderna: *a busca é um operador de melhoria de política*.

Intuição

A pergunta de fundo da aula: se eu tenho um simulador do mundo, quanto vale *pensar mais* antes de agir — e como converter esse pensamento em aprendizado permanente?

Onde estamos no curso

Da política global ao planejamento local



- ▶ Hoje aparece um recurso que ainda não tínhamos usado de forma séria: **um modelo do qual se pode amostrar**.
- ▶ E uma ideia que reaparece hoje em contextos muito além de jogos: **gastar computação em tempo de decisão** (*test-time compute*).

SEÇÃO 1

Busca baseada em simulação

01

Calcular a ação apenas para o estado atual

Até aqui, quase tudo que fizemos calcula um objeto *global*: $V^*(s)$ para todo s , ou $\pi_\theta(a \mid s)$ para toda entrada.

Calcular a ação apenas para o estado atual

Até aqui, quase tudo que fizemos calcula um objeto *global*: $V^*(s)$ para todo s , ou $\pi_\theta(a \mid s)$ para toda entrada.

Intuição

Ideia central da aula. Eu não preciso de uma boa política em todo lugar. Preciso de uma boa ação *no estado em que estou*. Se sobra computação, gaste-a localmente, no subproblema que começa agora.

Calcular a ação apenas para o estado atual

Até aqui, quase tudo que fizemos calcula um objeto *global*: $V^*(s)$ para todo s , ou $\pi_\theta(a \mid s)$ para toda entrada.

Intuição

Ideia central da aula. Eu não preciso de uma boa política em todo lugar. Preciso de uma boa ação *no estado em que estou*. Se sobra computação, gaste-a localmente, no subproblema que começa agora.

- ▶ O agente está em s_t (o estado *real*). Antes de agir, ele roda simulações internas a partir de s_t usando um modelo $\mathcal{M}$.
- ▶ Só a raiz importa: a busca produz a_t , o agente executa a_t no mundo real, observa s_{t+1} , e recomeça.
- ▶ Isso é **planejamento em tempo de decisão** — oposto de *planejamento em segundo plano*, que era o que a iteração de valor fazia.

O que exatamente é “ter um modelo”

Definição — Modelo generativo (simulador)

Caixa-preta $\mathcal{M}$ que, dado (s, a) , devolve uma *amostra* $s' \sim P(\cdot \mid s, a)$ e $r \sim R(\cdot \mid s, a)$. Não é preciso conhecer as probabilidades, nem varrer todos os sucessores.

O que exatamente é “ter um modelo”

Definição — Modelo generativo (simulador)

Caixa-preta $\mathcal{M}$ que, dado (s, a) , devolve uma *amostra* $s' \sim P(\cdot \mid s, a)$ e $r \sim R(\cdot \mid s, a)$. Não é preciso conhecer as probabilidades, nem varrer todos os sucessores.

- ▶ Em jogos de tabuleiro o simulador é *perfeito e gratuito*: as regras são o modelo.
- ▶ Em robótica e saúde ele é aprendido e imperfeito — e o erro se acumula ao longo do horizonte simulado.
- ▶ MCTS é agnóstico: exige apenas amostras. Por isso funciona com simuladores que ninguém sabe escrever em forma fechada.

O que exatamente é “ter um modelo”

Definição — Modelo generativo (simulador)

Caixa-preta $\mathcal{M}$ que, dado (s, a) , devolve uma *amostra* $s' \sim P(\cdot \mid s, a)$ e $r \sim R(\cdot \mid s, a)$. Não é preciso conhecer as probabilidades, nem varrer todos os sucessores.

- ▶ Em jogos de tabuleiro o simulador é *perfeito e gratuito*: as regras são o modelo.
- ▶ Em robótica e saúde ele é aprendido e imperfeito — e o erro se acumula ao longo do horizonte simulado.
- ▶ MCTS é agnóstico: exige apenas amostras. Por isso funciona com simuladores que ninguém sabe escrever em forma fechada.

Atenção

Um modelo bom para *um passo* pode ser inútil para *cinquenta*. A busca vale até onde o modelo ainda é fiel.

Busca de Monte Carlo simples

Dados um modelo $\mathcal{M}$ e uma *política de simulação* π :

Algoritmo — Busca de Monte Carlo simples

- ▶ Para cada ação $a \in \mathcal{A}$, simule K episódios a partir do estado real s_t : $\{s_t, a, r_{t+1}^k, s_{t+1}^k, \dots, s_T^k\}_{k=1}^K \sim \mathcal{M}, \pi$, e estime

$$\hat{Q}(s_t, a) = \frac{1}{K} \sum_{k=1}^K G_t^k \xrightarrow{K \rightarrow \infty} Q^\pi(s_t, a).$$

- ▶ Execute no mundo real $a_t = \arg \max_{a \in \mathcal{A}} \hat{Q}(s_t, a)$.

Busca de Monte Carlo simples

Dados um modelo $\mathcal{M}$ e uma *política de simulação* π :

Algoritmo — Busca de Monte Carlo simples

- ▶ Para cada ação $a \in \mathcal{A}$, simule K episódios a partir do estado real s_t :
 $\{s_t, a, r_{t+1}^k, s_{t+1}^k, \dots, s_T^k\}_{k=1}^K \sim \mathcal{M}, \pi$, e estime

$$\hat{Q}(s_t, a) = \frac{1}{K} \sum_{k=1}^K G_t^k \xrightarrow{K \rightarrow \infty} Q^\pi(s_t, a).$$

- ▶ Execute no mundo real $a_t = \arg \max_{a \in \mathcal{A}} \hat{Q}(s_t, a)$.

Intuição

Isso é **um passo de melhoria de política** sobre π , feito por amostragem, apenas no estado s_t .

Por que é exatamente um passo de melhoria

Seja π' a política que age gulosamente sobre Q^π em s_t e segue π depois. Pelo teorema de melhoria de política (Aula 3),

$$V^{\pi'}(s_t) = \max_a Q^\pi(s_t, a) \geq V^\pi(s_t).$$

Por que é exatamente um passo de melhoria

Seja π' a política que age gulosamente sobre Q^π em s_t e segue π depois. Pelo teorema de melhoria de política (Aula 3),

$$V^{\pi'}(s_t) = \max_a Q^\pi(s_t, a) \geq V^\pi(s_t).$$

- ▶ A busca de Monte Carlo simples *estima* esse $\max_a Q^\pi(s_t, a)$ por amostragem: com $K \rightarrow \infty$ recuperamos π' exatamente.
- ▶ O teto é claro: o resultado nunca é melhor que π' . Se π é ruim, um passo de melhoria continua sendo pouco.
- ▶ Os episódios abaixo do primeiro nível são *desperdiçados* como fonte de decisão: só servem para avaliar π .

Por que é exatamente um passo de melhoria

Seja π' a política que age gulosamente sobre Q^π em s_t e segue π depois. Pelo teorema de melhoria de política (Aula 3),

$$V^{\pi'}(s_t) = \max_a Q^\pi(s_t, a) \geq V^\pi(s_t).$$

- ▶ A busca de Monte Carlo simples *estima* esse $\max_a Q^\pi(s_t, a)$ por amostragem: com $K \rightarrow \infty$ recuperamos π' exatamente.
- ▶ O teto é claro: o resultado nunca é melhor que π' . Se π é ruim, um passo de melhoria continua sendo pouco.
- ▶ Os episódios abaixo do primeiro nível são *desperdiçados* como fonte de decisão: só servem para avaliar π .

Atenção

Além disso $\hat{Q}$ é média de retornos — variância alta. O $\arg \max$ de estimativas ruidosas introduz **viés de maximização**: escolhe-se a ação *com mais sorte*, não a melhor.

Podemos fazer melhor que um passo?

- ▶ Sim: se temos o modelo, podemos aplicar *repetidamente* o backup de Bellman ao longo da simulação — ou seja, buscar em profundidade, maximizando em cada nível.
- ▶ O objeto ideal é a **árvore expectimax**: alterna níveis de $\max$ sobre ações e de $\mathbb{E}$ sobre transições.

Podemos fazer melhor que um passo?

- ▶ Sim: se temos o modelo, podemos aplicar *repetidamente* o backup de Bellman ao longo da simulação — ou seja, buscar em profundidade, maximizando em cada nível.
- ▶ O objeto ideal é a **árvore expectimax**: alterna níveis de $\max$ sobre ações e de $\mathbb{E}$ sobre transições.

Backup expectimax até o horizonte H

$$Q_h(s, a) = R(s, a) + \gamma \sum_{s'} P(s' \mid s, a) \underbrace{\max_{a'} Q_{h-1}(s', a')}_{V_{h-1}(s')}, \quad Q_0 \equiv 0.$$

Na raiz, $\arg \max_a Q_H(s_t, a)$ é ótima para o sub-MDP de H passos que começa em s_t .

Podemos fazer melhor que um passo?

- ▶ Sim: se temos o modelo, podemos aplicar *repetidamente* o backup de Bellman ao longo da simulação — ou seja, buscar em profundidade, maximizando em cada nível.
- ▶ O objeto ideal é a **árvore expectimax**: alterna níveis de $\max$ sobre ações e de $\mathbb{E}$ sobre transições.

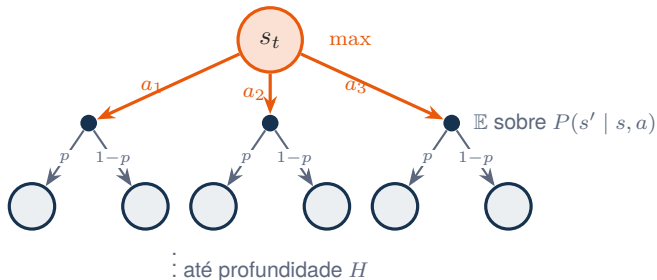
Backup expectimax até o horizonte H

$$Q_h(s, a) = R(s, a) + \gamma \sum_{s'} P(s' \mid s, a) \underbrace{\max_{a'} Q_{h-1}(s', a')}_{V_{h-1}(s')}, \quad Q_0 \equiv 0.$$

Na raiz, $\arg \max_a Q_H(s_t, a)$ é ótima para o sub-MDP de H passos que começa em s_t .

Não é preciso resolver o MDP inteiro — basta o *sub-MDP* que começa agora. Essa é a economia conceitual da busca dirigida para frente.

A árvore expectimax



- ▶ Nós circulares: estados (*esperança* sobre a dinâmica).
- ▶ Nós sólidos: ações (*maximização*).
- ▶ Em jogos de dois jogadores de soma zero, o nível do oponente vira um $\min$: a árvore expectimax degenera em *minimax*.

O problema: a árvore explode

Custo da enumeração

Uma árvore expectimax completa de profundidade H tem $(|\mathcal{S}| |\mathcal{A}|)^H$ nós.

O problema: a árvore explode

Custo da enumeração

Uma árvore expectimax completa de profundidade H tem $(|\mathcal{S}| |\mathcal{A}|)^H$ nós.

- ▶ Xadrez: fator de ramificação $b \approx 35$, partida típica $d \approx 80$.
- ▶ Go 19×19 : $b \approx 250$, $d \approx 150$. 250^{150} é maior que qualquer número físico relevante.
- ▶ Com poda alfa-beta (só válida em minimax determinístico) o expoente cai pela metade — ainda impossível.

O problema: a árvore explode

Custo da enumeração

Uma árvore expectimax completa de profundidade H tem $(|S| |\mathcal{A}|)^H$ nós.

- ▶ Xadrez: fator de ramificação $b \approx 35$, partida típica $d \approx 80$.
- ▶ Go 19×19 : $b \approx 250$, $d \approx 150$. 250^{150} é maior que qualquer número físico relevante.
- ▶ Com poda alfa-beta (só válida em minimax determinístico) o expoente cai pela metade — ainda impossível.

Intuição

Duas saídas clássicas — e o MCTS usa **as duas**:

- ▶ **Amostrar em vez de enumerar** — troca a soma sobre s' por amostras (quebra a maldição da dimensionalidade da *largura*).
- ▶ **Truncar em vez de ir até o fim** — troca a subárvore abaixo de uma folha por uma *estimativa* de valor (maldição da *profundidade*).

Teste sua compreensão

Com orçamento fixo de N chamadas ao simulador, vale mais distribuir as simulações igualmente entre as $|\mathcal{A}|$ ações da raiz, ou concentrá-las nas que *até agora* parecem boas? Em que regime a resposta muda?

Pergunta para discussão conduzida em aula.

Para discutir

Teste sua compreensão

Com orçamento fixo de N chamadas ao simulador, vale mais distribuir as simulações igualmente entre as $|\mathcal{A}|$ ações da raiz, ou concentrá-las nas que *até agora* parecem boas? Em que regime a resposta muda?

Pergunta para discussão conduzida em aula.

Teste sua compreensão

Por que a poda alfa-beta, tão eficaz no xadrez dos anos 1990, não resolveu o *Go*? Que propriedade do problema ela exige e o *Go* não oferece?

Pergunta para discussão conduzida em aula.

SEÇÃO 2

MCTS e a regra UCT

02

Busca em árvore de Monte Carlo (MCTS)

Algoritmo — Esqueleto do MCTS

Dado um modelo $\mathcal{M}$ e um orçamento de K simulações:

1. Construa uma árvore de busca enraizada no estado atual s_t .
2. Repita K vezes: percorra a árvore *amostrando* ações e estados sucessores, expanda-a em um ponto, estime o valor da nova folha e propague a estimativa de volta até a raiz.
3. Ao fim do orçamento, escolha a ação real $a_t = \arg \max_{a \in \mathcal{A}} f(s_{\text{raiz}}, a)$, onde f é o valor estimado ou a contagem de visitas.

Busca em árvore de Monte Carlo (MCTS)

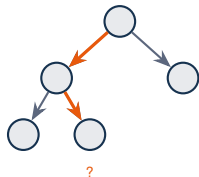
Algoritmo — Esqueleto do MCTS

Dado um modelo $\mathcal{M}$ e um orçamento de K simulações:

1. Construa uma árvore de busca enraizada no estado atual s_t .
2. Repita K vezes: percorra a árvore *amostrando* ações e estados sucessores, expanda-a em um ponto, estime o valor da nova folha e propague a estimativa de volta até a raiz.
3. Ao fim do orçamento, escolha a ação real $a_t = \arg \max_{a \in \mathcal{A}} f(s_{\text{raiz}}, a)$, onde f é o valor estimado ou a contagem de visitas.

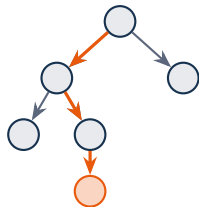
- ▶ A árvore é **assimétrica**: cresce fundo onde parece promissor e raso onde não parece. Isso é o oposto da busca em largura.
- ▶ É **anytime**: pode ser interrompida a qualquer momento e devolve a melhor decisão encontrada até ali.

As quatro fases de uma simulação



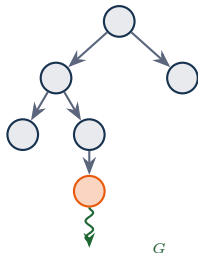
1. Seleção

desce pela árvore com a política de árvore (UCT)



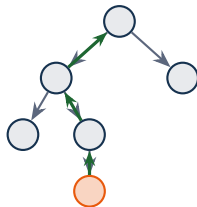
2. Expansão

acrescenta um nó novo ao encontrar ação não tentada



3. Simulação

estima o valor da folha: *rollout* ou rede de valor



4. Retropropagação

atualiza N e Q em todas as arestas do caminho

Intuição

Dentro da árvore a política *muda a cada simulação*. Fora dela, o *rollout* usa uma política fixa e barata.

O que fica guardado em cada aresta

Para cada par (i, a) — nó i da árvore, ação a — o algoritmo mantém:

Estatística	Significado	Atualização
$N(i, a)$	vezes que a foi escolhida em i	+1 a cada visita
$W(i, a)$	soma dos retornos observados	+ G
$\hat{Q}(i, a)$	valor médio $W(i, a)/N(i, a)$	recalculado
$N(i)$	$\sum_a N(i, a)$, visitas ao nó	+1

- ▶ São *médias de amostras* — nada de matriz de transição, nada de varredura sobre $\mathcal{S}$.
- ▶ A memória cresce com o número de nós *visitados*, não com $|\mathcal{S}|$. É por isso que MCTS roda em jogos com 10^{170} posições.

Cada nó é um problema de bandit

Escolher qual ramo simular é o dilema exploração/aproveitamento da Aula 10:

- ▶ cada ação a no nó i é um *braço*;
- ▶ o “ganho” é o retorno G da simulação que passa por ele;
- ▶ queremos gastar simulações nos braços bons, sem descartar cedo demais um braço que teve azar.

Cada nó é um problema de bandit

Escolher qual ramo simular é o dilema exploração/aproveitamento da Aula 10:

- ▶ cada ação a no nó i é um *braço*;
- ▶ o “ganho” é o retorno G da simulação que passa por ele;
- ▶ queremos gastar simulações nos braços bons, sem descartar cedo demais um braço que teve azar.

UCB1 (Auer, Cesa-Bianchi e Fischer, 2002) — revisão

$$a_t = \arg \max_a \hat{Q}(a) + c \sqrt{\frac{\ln t}{N(a)}}.$$

O segundo termo é a largura de um intervalo de confiança: encolhe como $1/\sqrt{N(a)}$ e cresce com o tempo, de modo que nenhum braço é abandonado para sempre.

Cada nó é um problema de bandit

Escolher qual ramo simular é o dilema exploração/aproveitamento da Aula 10:

- ▶ cada ação a no nó i é um *braço*;
- ▶ o “ganho” é o retorno G da simulação que passa por ele;
- ▶ queremos gastar simulações nos braços bons, sem descartar cedo demais um braço que teve azar.

UCB1 (Auer, Cesa-Bianchi e Fischer, 2002) — revisão

$$a_t = \arg \max_a \hat{Q}(a) + c \sqrt{\frac{\ln t}{N(a)}}.$$

O segundo termo é a largura de um intervalo de confiança: encolhe como $1/\sqrt{N(a)}$ e cresce com o tempo, de modo que nenhum braço é abandonado para sempre.

UCT = UCB aplicado a árvores: use UCB1 *em cada nó*.

UCT: *Upper Confidence Trees*

Regra de seleção UCT (Kocsis e Szepesvári, 2006)

No nó i , selecione

$$a = \arg \max_{a \in \mathcal{A}(i)} \hat{Q}(i, a) + c \sqrt{\frac{\ln N(i)}{N(i, a)}} \quad \hat{Q}(i, a) = \frac{1}{N(i, a)} \sum_{k=1}^{N(i, a)} G^k(i, a)$$

com $G^k(i, a)$ o k -ésimo retorno a partir de i após a e $N(i) = \sum_a N(i, a)$.

UCT: Upper Confidence Trees

Regra de seleção UCT (Kocsis e Szepesvári, 2006)

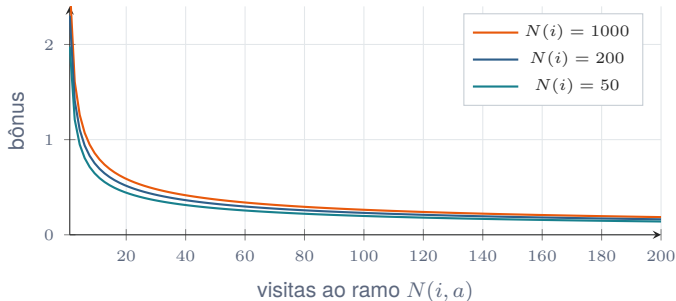
No nó i , selecione

$$a = \arg \max_{a \in \mathcal{A}(i)} \hat{Q}(i, a) + c \sqrt{\frac{\ln N(i)}{N(i, a)}} \quad \hat{Q}(i, a) = \frac{1}{N(i, a)} \sum_{k=1}^{N(i, a)} G^k(i, a)$$

com $G^k(i, a)$ o k -ésimo retorno a partir de i após a e $N(i) = \sum_a N(i, a)$.

- ▶ Ação nunca tentada tem $N(i, a) = 0$: bônus infinito — é visitada primeiro (a *expansão*).
- ▶ c depende da escala dos retornos: normalizar G é padrão.
- ▶ Como as estatísticas mudam, a política de árvore **muda a cada simulação** — há melhoria de política *durante* a busca.

Como o bônus decai



- ▶ O bônus cai como $N(i, a)^{-1/2}$ — rápido no começo, lento depois.
- ▶ Cresce como $\sqrt{\ln N(i)}$: dobrar o orçamento total mal move a curva. É por isso que MCTS *concentra* o esforço.
- ▶ Consequência prática: a distribuição de visitas na raiz é fortemente desbalanceada, e é justamente isso que a torna informativa.

MCTS/UCT em pseudocódigo

Algoritmo — UCT

entrada: s_t , modelo $\mathcal{M}$, orçamento K , constante c ; $\text{árvore} \leftarrow \text{raiz } s_{\text{raiz}} = s_t$

para $k = 1, \dots, K$:

$s \leftarrow s_{\text{raiz}}$; $\text{caminho} \leftarrow []$

enquanto s está na árvore e todas as ações de s já foram tentadas:

$a \leftarrow \arg \max_a \hat{Q}(s, a) + c\sqrt{\ln N(s)/N(s, a)}$

seleção

$(s', r) \sim \mathcal{M}(s, a)$; anexe (s, a, r) ao caminho; $s \leftarrow s'$

se s não é terminal: acrescente s à árvore

expansão

$G \leftarrow \text{AVALIA}(s)$

simulação

para (s, a, r) no caminho, de trás para frente:

$G \leftarrow r + \gamma G$; $N(s, a) += 1$; $W(s, a) += G$; $\hat{Q}(s, a) \leftarrow W(s, a)/N(s, a)$

retropropagação

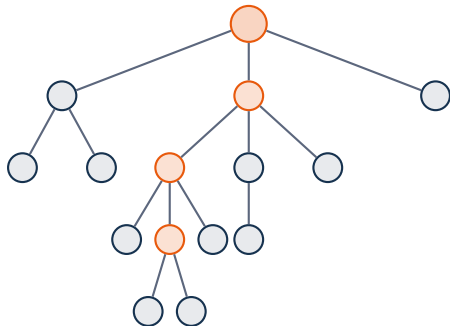
devolva $\arg \max_a N(s_{\text{raiz}}, a)$

$\text{AVALIA}(s)$: *rollout* até o fim (MCTS clássico) ou $v_\theta(s)$ de uma rede de valor (AlphaZero).

Detalhes que decidem se funciona

- ▶ **Ação final: visitas ou valor?** A prática consagrada é $\arg \max_a N(s_{\text{raiz}}, a)$, não $\arg \max_a \hat{Q}(s_{\text{raiz}}, a)$. A contagem de visitas é uma média sobre todo o histórico da busca — muito mais robusta a um único retorno afortunado.
- ▶ **Reaproveitamento da árvore.** Depois de agir, a subárvore correspondente ao novo estado já está construída: mantenha-a e descarte o resto.
- ▶ **Dois jogadores.** O retorno deve trocar de sinal a cada nível — o que é bom para mim é ruim para o adversário. Um erro de sinal aqui produz um agente que joga bem *contra si mesmo* e mal contra qualquer outro.
- ▶ **Transposições.** Estados alcançáveis por ordens diferentes de jogadas podem compartilhar estatísticas (grafo em vez de árvore).
- ▶ **Normalização de G .** Sem ela, c deixa de ter significado e o bônus vira ruído ou tirania.

A árvore que o UCT constrói



- ▶ Em laranja, o caminho principal (*principal variation*) — onde a busca gastou quase todo o orçamento.
- ▶ Ramos ruins são visitados uma ou duas vezes e abandonados; ramos promissores são aprofundados. **Busca best-first altamente seletiva.**

O que se pode provar

Consistência do UCT (Kocsis e Szepesvári, 2006)

Sob condições adequadas (retornos limitados, exploração suficiente), a probabilidade de o UCT escolher uma ação subótima na raiz tende a zero quando o número de simulações $K \rightarrow \infty$; a estimativa na raiz converge para o valor ótimo do sub-MDP.

O que se pode provar

Consistência do UCT (Kocsis e Szepesvári, 2006)

Sob condições adequadas (retornos limitados, exploração suficiente), a probabilidade de o UCT escolher uma ação subótima na raiz tende a zero quando o número de simulações $K \rightarrow \infty$; a estimativa na raiz converge para o valor ótimo do sub-MDP.

Atenção

Leia a letra miúda. A garantia é *assintótica*, e a taxa de convergência pode ser **duplamente exponencial na profundidade** no pior caso (Coquelin e Munos, 2007). Em problemas com uma única sequência estreita de ações boas — uma “agulha no palheiro” — o UCT é catastroficamente lento.

O que se pode provar

Consistência do UCT (Kocsis e Szepesvári, 2006)

Sob condições adequadas (retornos limitados, exploração suficiente), a probabilidade de o UCT escolher uma ação subótima na raiz tende a zero quando o número de simulações $K \rightarrow \infty$; a estimativa na raiz converge para o valor ótimo do sub-MDP.

Atenção

Leia a letra miúda. A garantia é *assintótica*, e a taxa de convergência pode ser **duplamente exponencial na profundidade** no pior caso (Coquelin e Munos, 2007). Em problemas com uma única sequência estreita de ações boas — uma “agulha no palheiro” — o UCT é catastroficamente lento.

- ▶ Ou seja: a teoria diz que *eventualmente* funciona; a prática funciona porque o domínio costuma ser mais benigno que o pior caso e porque priors bons (Seção 3) encurtam drasticamente a busca.

Vantagens do MCTS

- ▶ **Best-first seletivo** — gasta onde importa.
- ▶ **Avaliação dinâmica** — estima o valor do estado *atual*, não de todos.
- ▶ **Amostragem** quebra a maldição da dimensionalidade.
- ▶ **Modelo caixa-preta** — só precisa de amostras.
- ▶ **Anytime** — interrompa quando quiser.
- ▶ **Paralelizável** — simulações são quase independentes (com cuidado: *virtual loss*).
- ▶ **Sem heurística de domínio** — dispensa a função de avaliação à mão do alfa-beta.
- ▶ **Melhora com computação** — mais tempo, melhor jogada, sem retreinar nada.

Intuição

A última é a mais subestimada: MCTS converte *computação em tempo de decisão* em *qualidade de decisão* — a propriedade que hoje se busca em modelos de raciocínio.

Limitações — e o que se faz a respeito

Limitação	Remédio usual
Ramificação enorme	<i>prior</i> de política que concentra a busca (PUCT, Seção 3)
Ações contínuas <i>Rollouts</i> aleatórios são fracos	<i>progressive widening</i> : filhos crescem como N^α substituí-los por uma rede de valor v_θ
Poucos dados por nó	compartilhar estatísticas entre nós (RAVE / AMAF)
Modelo aprendido, imperfeito	horizonte curto + valor aprendido, ou aprender o modelo no espaço de <i>valores</i> (MuZero)

Atenção

MCTS pressupõe um simulador *fiel* e *barato*. Onde uma dessas condições falha — robótica, saúde, educação — ele deixa de ser a ferramenta óbvia.

Para discutir

Teste sua compreensão

O UCT devolve $\arg \max_a N(s_{\text{raiz}}, a)$ em vez de $\arg \max_a \hat{Q}(s_{\text{raiz}}, a)$. Construa um exemplo pequeno em que as duas regras discordam. Qual das duas você confiaria mais, e por quê?

Pergunta para discussão conduzida em aula.

Para discutir

Teste sua compreensão

O UCT devolve $\arg \max_a N(s_{\text{raiz}}, a)$ em vez de $\arg \max_a \hat{Q}(s_{\text{raiz}}, a)$. Construa um exemplo pequeno em que as duas regras discordam. Qual das duas você confiaria mais, e por quê?

Pergunta para discussão conduzida em aula.

Teste sua compreensão

MCTS é “eficiente em computação” mas cada decisão exige milhares de chamadas ao simulador. Em que sentido, então, ele é *eficiente em amostras* — e em que sentido não é? Como isso se compara com o que vimos nas Aulas 11–12?

Pergunta para discussão conduzida em aula.

SEÇÃO 3

De AlphaGo a MuZero

03

Por que o Go era o problema-símbolo

- ▶ Tabuleiro 19×19 ; cerca de 10^{170} posições legais — mais que átomos no universo observável, por larga margem.
- ▶ Ramificação $b \approx 250$ contra $b \approx 35$ do xadrez.
- ▶ Não existe boa função de avaliação escrita à mão: o valor de uma posição depende de padrões globais sutis (vida e morte, influência, *ko*).
- ▶ Por isso o alfa-beta com heurística — receita do Deep Blue, 1997 — **não transferiu**.

Intuição

A partir de 2006, MCTS levou os programas de Go do nível amador fraco ao amador forte. Faltava ainda uma *avaliação de posição* confiável — e é exatamente isso que redes profundas passaram a fornecer.

Atenção

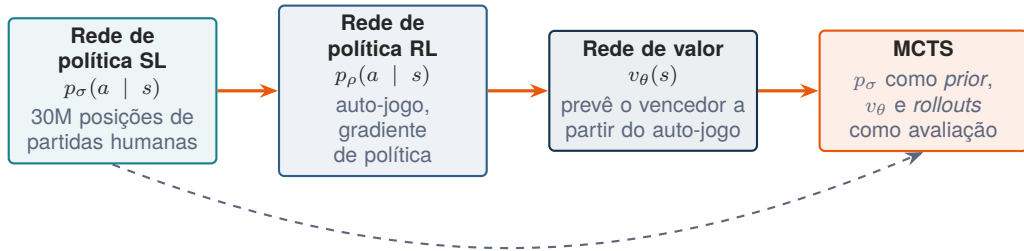
Previsões de especialistas em 2014 colocavam o Go em nível profissional a “uma década de distância”. Levou dois anos.

A linha do tempo



- ▶ O fio condutor é uma **remoção sucessiva de conhecimento humano**: primeiro os jogos de mestres, depois as *features* de domínio, depois os *rollouts*, por fim as próprias regras.
- ▶ O que sobra em MuZero é: um simulador de si mesmo, uma busca e um sinal de recompensa.

AlphaGo (2016): três redes e uma busca



- ▶ Avaliação de folha por **mistura**: $\hat{V}(s_{\text{folha}}) = (1 - \lambda) v_{\theta}(s_{\text{folha}}) + \lambda z_{\text{rollout}}$. As duas fontes erram de maneiras diferentes; a média erra menos.
- ▶ Curiosamente, a rede *SL* funcionava melhor como *prior* da busca do que a rede *RL* — ela é mais *diversa*, e a busca precisa de diversidade.
- ▶ Vitórias: Fan Hui 5–0 (2015) e Lee Sedol 4–1 (2016).

AlphaGo Zero (2017): o que foi *removido*

	AlphaGo	AlphaGo Zero
Partidas humanas	sim, 30M posições	não
<i>Features</i> de domínio	sim, dezenas	só o tabuleiro
<i>Rollouts</i> na busca	sim	não
Redes separadas	três	uma, com duas cabeças
Arquitetura	convolucional	residual

Definição — Rede de duas cabeças

$(p, v) = f_{\theta}(s)$, com $p \in \Delta(\mathcal{A})$ o *prior* de política e $v \in [-1, 1]$ o valor da posição.

- ▶ Compartilhar o tronco não é só economia: as tarefas *regularizam* uma à outra.
- ▶ Derrotou por 100–0 a versão que venceu Lee Sedol, após três dias de treino a partir de pesos aleatórios.

PUCT: o *prior* entra na regra de seleção

Regra de seleção do AlphaGo Zero / AlphaZero

$$a = \arg \max_a Q(s, a) + c_{\text{puct}} P(s, a) \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)}, \quad P(s, a) = p_a, \quad Q = \frac{W}{N}.$$

PUCT: o *prior* entra na regra de seleção

Regra de seleção do AlphaGo Zero / AlphaZero

$$a = \arg \max_a Q(s, a) + c_{\text{puct}} P(s, a) \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)}, \quad P(s, a) = p_a, \quad Q = \frac{W}{N}.$$

Três diferenças em relação ao UCT:

- ▶ o bônus é **multiplicado pelo *prior*** $P(s, a)$: ações implausíveis para a rede quase não são exploradas;
- ▶ o decaimento $1/(1 + N(s, a))$ é mais agressivo que $1/\sqrt{N(s, a)}$;
- ▶ não há $\ln$: cresce com $\sqrt{\cdot}$ no total de visitas.

PUCT: o *prior* entra na regra de seleção

Regra de seleção do AlphaGo Zero / AlphaZero

$$a = \arg \max_a Q(s, a) + c_{\text{puct}} P(s, a) \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)}, \quad P(s, a) = p_a, \quad Q = \frac{W}{N}.$$

Três diferenças em relação ao UCT:

- ▶ o bônus é **multiplicado pelo *prior*** $P(s, a)$: ações implausíveis para a rede quase não são exploradas;
- ▶ o decaimento $1/(1 + N(s, a))$ é mais agressivo que $1/\sqrt{N(s, a)}$;
- ▶ não há $\ln$: cresce com $\sqrt{\cdot}$ no total de visitas.

Intuição

Com $b \approx 250$, é o *prior* que torna a busca viável: ela olha a sério para meia dúzia de jogadas. A rede não substitui a busca — poda-a.

A ideia central: a busca é um operador de melhoria de política

Ao fim de K simulações na raiz, defina a **política da busca**

$$\pi_{\text{MCTS}}(a \mid s) = \frac{N(s, a)^{1/\tau}}{\sum_b N(s, b)^{1/\tau}}, \quad \tau > 0 \text{ (temperatura).}$$

A ideia central: a busca é um operador de melhoria de política

Ao fim de K simulações na raiz, defina a **política da busca**

$$\pi_{\text{MCTS}}(a \mid s) = \frac{N(s, a)^{1/\tau}}{\sum_b N(s, b)^{1/\tau}}, \quad \tau > 0 \text{ (temperatura)}.$$

O laço de AlphaZero

Empiricamente, π_{MCTS} é *mais forte* que o *prior* p da rede que a gerou. Então: use π_{MCTS} como **alvo de treinamento** para p , e o resultado da partida z como alvo para v . A rede melhorada torna a próxima busca melhor, que produz alvos ainda melhores.

A ideia central: a busca é um operador de melhoria de política

Ao fim de K simulações na raiz, defina a **política da busca**

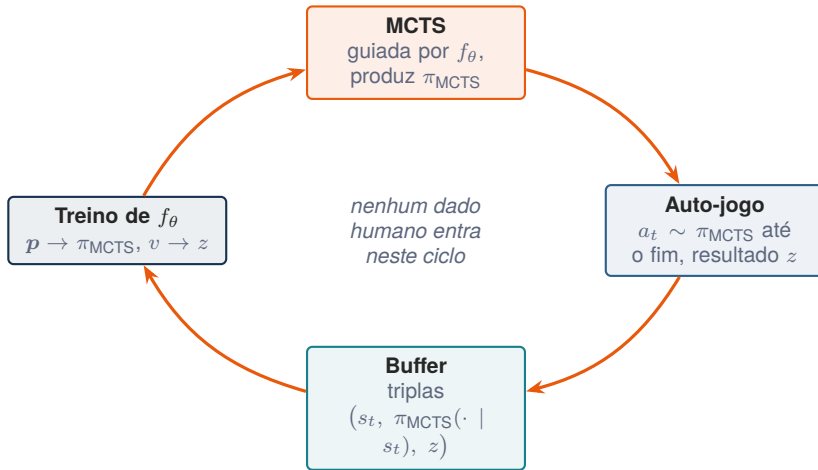
$$\pi_{\text{MCTS}}(a \mid s) = \frac{N(s, a)^{1/\tau}}{\sum_b N(s, b)^{1/\tau}}, \quad \tau > 0 \text{ (temperatura).}$$

O laço de AlphaZero

Empiricamente, π_{MCTS} é *mais forte* que o *prior* p da rede que a gerou. Então: use π_{MCTS} como **alvo de treinamento** para p , e o resultado da partida z como alvo para v . A rede melhorada torna a próxima busca melhor, que produz alvos ainda melhores.

- ▶ Isto é **iteração de política** (Aula 3), com a busca no papel do passo de *melhoria* e o auto-jogo no papel da *avaliação*.
- ▶ Também é chamado de *expert iteration* (Anthony, Tian e Barber, 2017): a busca é o “especialista lento”, a rede é o “aprendiz rápido” que *amortiza* o especialista.

O laço de auto-jogo



A perda de treinamento

Objetivo do AlphaGo Zero / AlphaZero

Para cada posição s amostrada do auto-jogo, com alvos $\pi = \pi_{\text{MCTS}}(\cdot \mid s)$ e $z \in \{-1, 0, +1\}$:

$$\ell(\theta) = \underbrace{(z - v_{\theta}(s))^2}_{\text{regressão de valor}} - \underbrace{\pi^{\top} \log \mathbf{p}_{\theta}(s)}_{\text{destilação da busca}} + \underbrace{c \|\theta\|^2}_{\text{regularização}} .$$

A perda de treinamento

Objetivo do AlphaGo Zero / AlphaZero

Para cada posição s amostrada do auto-jogo, com alvos $\pi = \pi_{\text{MCTS}}(\cdot \mid s)$ e $z \in \{-1, 0, +1\}$:

$$\ell(\theta) = \underbrace{(z - v_{\theta}(s))^2}_{\text{regressão de valor}} - \underbrace{\pi^{\top} \log \mathbf{p}_{\theta}(s)}_{\text{destilação da busca}} + \underbrace{c \|\theta\|^2}_{\text{regularização}}.$$

- ▶ O segundo termo é *entropia cruzada*: aprendizado supervisionado puro, onde o rótulo foi fabricado pela própria busca.
- ▶ O primeiro termo é o que fecha o ciclo: sem um v bom, a busca degrada e os rótulos pioram.
- ▶ Note o que *não* aparece: nenhum gradiente de política, razão de importância ou região de confiança — o problema difícil de RL foi empurrado para dentro da busca.

Duas engenhocas indispensáveis

Definição — Ruído de Dirichlet na raiz

$$P(s_{\text{raiz}}, a) = (1 - \varepsilon) p_a + \varepsilon \eta_a, \quad \boldsymbol{\eta} \sim \text{Dir}(\boldsymbol{\alpha}), \quad \varepsilon \approx 0,25.$$

Garante que toda jogada legal tenha alguma chance de ser explorada, mesmo que a rede a considere impossível. Sem isso, o auto-jogo colapsa em um repertório estreito.

Duas engenhocas indispensáveis

Definição — Ruído de Dirichlet na raiz

$$P(s_{\text{raiz}}, a) = (1 - \varepsilon) p_a + \varepsilon \eta_a, \quad \eta \sim \text{Dir}(\alpha), \quad \varepsilon \approx 0,25.$$

Garante que toda jogada legal tenha alguma chance de ser explorada, mesmo que a rede a considere impossível. Sem isso, o auto-jogo colapsa em um repertório estreito.

Definição — Temperatura τ

Nas primeiras jogadas, $\tau = 1$ (amostra proporcional às visitas), o que gera **diversidade de aberturas**; depois $\tau \rightarrow 0$ (praticamente $\arg \max$), jogo forte.

Duas engenhocas indispensáveis

Definição — Ruído de Dirichlet na raiz

$$P(s_{\text{raiz}}, a) = (1 - \varepsilon) p_a + \varepsilon \eta_a, \quad \eta \sim \text{Dir}(\alpha), \quad \varepsilon \approx 0,25.$$

Garante que toda jogada legal tenha alguma chance de ser explorada, mesmo que a rede a considere impossível. Sem isso, o auto-jogo colapsa em um repertório estreito.

Definição — Temperatura τ

Nas primeiras jogadas, $\tau = 1$ (amostra proporcional às visitas), o que gera **diversidade de aberturas**; depois $\tau \rightarrow 0$ (praticamente $\arg \max$), jogo forte.

Atenção

As duas são a versão de auto-jogo do problema de *exploração* das Aulas 10–12: sem elas o sistema aprende a bater apenas em si mesmo.

AlphaZero (2018): a mesma receita, três jogos

- ▶ Um único algoritmo, sem conhecimento específico de domínio além das regras, treinado do zero por auto-jogo em **Go**, **xadrez** e **shogi**.
- ▶ Derrotou o Stockfish (xadrez), o elmo (shogi) e a versão anterior do AlphaGo Zero.
- ▶ Avaliava algo em torno de *dezenas de milhares* de posições por jogada, contra as *dezenas de milhões* do Stockfish — e ainda assim jogava melhor.

AlphaZero (2018): a mesma receita, três jogos

- ▶ Um único algoritmo, sem conhecimento específico de domínio além das regras, treinado do zero por auto-jogo em **Go**, **xadrez** e **shogi**.
- ▶ Derrotou o Stockfish (xadrez), o elmo (shogi) e a versão anterior do AlphaGo Zero.
- ▶ Avaliava algo em torno de *dezenas de milhares* de posições por jogada, contra as *dezenas de milhões* do Stockfish — e ainda assim jogava melhor.

Intuição

A lição não é “busca” nem “rede”, é a **divisão de trabalho**: a rede fornece julgamento posicional (largura), a busca fornece cálculo concreto (profundidade). Um bom *prior* vale por várias ordens de grandeza de simulações.

AlphaZero (2018): a mesma receita, três jogos

- ▶ Um único algoritmo, sem conhecimento específico de domínio além das regras, treinado do zero por auto-jogo em **Go**, **xadrez** e **shogi**.
- ▶ Derrotou o Stockfish (xadrez), o elmo (shogi) e a versão anterior do AlphaGo Zero.
- ▶ Avaliava algo em torno de *dezenas de milhares* de posições por jogada, contra as *dezenas de milhões* do Stockfish — e ainda assim jogava melhor.

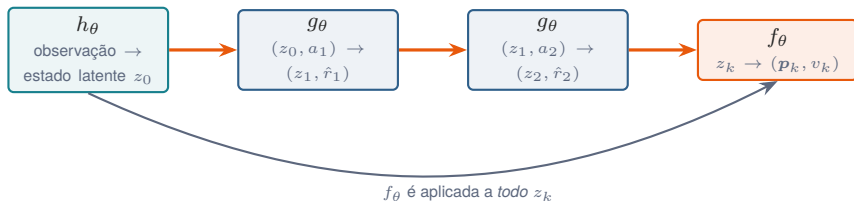
Intuição

A lição não é “busca” nem “rede”, é a **divisão de trabalho**: a rede fornece julgamento posicional (largura), a busca fornece cálculo concreto (profundidade). Um bom *prior* vale por várias ordens de grandeza de simulações.

Atenção

Comparações de força entre AlphaZero e motores clássicos dependem de tempo por jogada, *hardware* e livro de aberturas: a conclusão qualitativa é sólida, os números exatos nem tanto.

MuZero (2020): e se nem as regras forem dadas?



- ▶ Não há decodificador: z_k **não** precisa reconstruir o estado real. Basta que prediga *recompensa, valor e política* — exatamente o que a busca consome.
- ▶ A busca roda inteiramente no espaço latente. O modelo é aprendido *para servir ao planejamento*, não para ser fiel.
- ▶ Igualou AlphaZero nos jogos de tabuleiro e estabeleceu o estado da arte em Atari, onde ninguém tem as regras.

Para discutir

Teste sua compreensão

O alvo de política é a distribuição de visitas π , e o alvo de valor é o resultado final z da partida. Por que não usar $\hat{Q}(s_{\text{raiz}}, a)$ da busca como alvo de valor, já que ele parece mais informativo?

Pergunta para discussão conduzida em aula.

Para discutir

Teste sua compreensão

O alvo de política é a distribuição de visitas π , e o alvo de valor é o resultado final z da partida. Por que não usar $\hat{Q}(s_{\text{raiz}}, a)$ da busca como alvo de valor, já que ele parece mais informativo?

Pergunta para discussão conduzida em aula.

Teste sua compreensão

O auto-jogo em jogos de dois jogadores de soma zero gera automaticamente um adversário do próprio nível — um currículo. Que ingrediente equivalente existiria em um problema de um jogador só, como controle de um robô?

Pergunta para discussão conduzida em aula.

O que generaliza — e o que não

Transfere bem

- ▶ Busca como operador de melhoria; rede como amortização da busca.
- ▶ *Prior* de política para podar ramificação alta.
- ▶ Trocar computação de decisão por qualidade de decisão.
- ▶ Aprender o modelo no espaço do que o planejador consome.

Não transfere de graça

- ▶ Simulador perfeito, determinístico e barato.
- ▶ Recompensa terminal não ambígua (± 1).
- ▶ Auto-jogo como fonte gratuita de currículo.
- ▶ Estados episódicos e reinicializáveis à vontade.

Atenção

Fora de jogos o gargalo raramente é a busca: é ter um modelo confiável por vinte passos e uma recompensa que signifique o que queremos.

- ▶ **Planejamento e controle** — MPC amostral, POMDPs (POMCP, DESPOT), escalonamento, roteamento, síntese de programas.
- ▶ **Química e matemática** — retrossíntese, descoberta de algoritmos (AlphaTensor), demonstração de teoremas (AlphaProof).
- ▶ **Modelos de linguagem** — busca em árvore sobre passos de raciocínio, com o próprio modelo como *prior* e um verificador no papel da rede de valor. O laço “gerar candidatos, avaliar, destilar de volta no modelo” é o laço de AlphaZero.

- ▶ **Planejamento e controle** — MPC amostral, POMDPs (POMCP, DESPOT), escalonamento, roteamento, síntese de programas.
- ▶ **Química e matemática** — retrossíntese, descoberta de algoritmos (AlphaTensor), demonstração de teoremas (AlphaProof).
- ▶ **Modelos de linguagem** — busca em árvore sobre passos de raciocínio, com o próprio modelo como *prior* e um verificador no papel da rede de valor. O laço “gerar candidatos, avaliar, destilar de volta no modelo” é o laço de AlphaZero.

Atenção

A analogia tem limite: em texto, o “estado” não é markoviano de forma óbvia, o ramo é essencialmente infinito e não existe uma regra que declare vitória. É por isso que a peça mais difícil dessas construções é o *verificador*, não a busca.

SEÇÃO 4

Fechamento

04

1. **Planejamento em tempo de decisão.** Com um simulador, vale a pena gastar computação local para decidir melhor *agora*.
2. **Busca de Monte Carlo simples** = um passo de melhoria de política. A árvore expectimax faria melhor, mas custa $(|\mathcal{S}||\mathcal{A}|)^H$.
3. **MCTS** escapa amostrando (largura) e truncando (profundidade), e constrói uma árvore assimétrica, *anytime*.
4. **UCT** trata cada nó como um bandit; converge, mas com garantias fracas no pior caso.
5. **AlphaZero** substitui *rollouts* por v_θ , guia a busca com p_θ via PUCT, e treina a rede nos alvos que a própria busca produz — iteração de política com a busca no lugar do $\max$.
6. **MuZero** aprende também o modelo, no espaço do que o planejador consome.

UCT

$$a = \arg \max_a \hat{Q}(i, a) + c \sqrt{\frac{\ln N(i)}{N(i, a)}}$$

PUCT

$$a = \arg \max_a Q(s, a) + c_{\text{puct}} P(s, a) \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)}$$

Política da busca

$$\pi_{\text{MCTS}}(a \mid s) \propto N(s, a)^{1/\tau}$$

Retropropagação

$$G \leftarrow r + \gamma G, \quad \hat{Q} \leftarrow \frac{W+G}{N+1}$$

Perda AlphaZero

$$\ell = (z - v)^2 - \boldsymbol{\pi}^\top \log \mathbf{p} + c \|\boldsymbol{\theta}\|^2$$

Ruído na raiz

$$P \leftarrow (1 - \varepsilon) \mathbf{p} + \varepsilon \boldsymbol{\eta}, \quad \boldsymbol{\eta} \sim \text{Dir}(\boldsymbol{\alpha})$$

Atenção

- ▶ Devolver $\arg \max_a \hat{Q}$ em vez de $\arg \max_a N(\cdot, a)$ — e ser traído por um único *rollout* sortudo.
- ▶ Esquecer de inverter o sinal do retorno a cada nível em jogos de dois jogadores.
- ▶ Usar c do UCT sem normalizar os retornos.
- ▶ Descartar a árvore inteira a cada jogada.
- ▶ Treinar p contra o $\arg \max$ da busca em vez da distribuição de visitas — perde-se toda a informação de incerteza.
- ▶ Supor que um modelo com 1% de erro por passo é utilizável em um horizonte de 50 passos.

Leituras

- ▶ L. Kocsis e C. Szepesvári. Bandit based Monte-Carlo planning. *ECML*, 2006. o artigo do UCT
- ▶ C. Browne et al. A survey of Monte Carlo tree search methods. *IEEE TCIAIG*, 4(1):1–43, 2012.
- ▶ P.-A. Coquelin e R. Munos. Bandit algorithms for tree search. *UAI*, 2007. os limites do UCT
- ▶ D. Silver et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529:484–489, 2016.
- ▶ D. Silver et al. Mastering the game of Go without human knowledge. *Nature*, 550:354–359, 2017.
- ▶ D. Silver et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362:1140–1144, 2018.
- ▶ T. Anthony, Z. Tian e D. Barber. Thinking fast and slow with deep learning and tree search. *NeurIPS*, 2017.
- ▶ J. Schrittwieser et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.
- ▶ R. Sutton e A. Barto. *Reinforcement Learning*, 2ª ed., Cap. 8.
- ▶ E. Brunskill. *CS234: Reinforcement Learning*, Stanford — *Lecture 13: Monte Carlo Tree Search*.

Próxima aula

- ▶ **Recompensas em RL:** de onde vem R , o que acontece quando ela está errada, e como especificá-la sem produzir comportamento indesejado.
- ▶ Guarde desta aula a pergunta que fica: MCTS otimiza *muito bem* o que lhe for dado como recompensa. Isso é uma virtude apenas se a recompensa estiver certa.

Material adaptado e expandido de **CS234: Reinforcement Learning**, Emma Brunskill, Stanford University.

Prof. Marcelo Keese Albertini — Faculdade de Computação, UFU.